

BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC NÔNG LÂM TP HCM

KHOA CÔNG NGHỆ THÔNG TIN



LẬP TRÌNH TRÊN THIẾT BỊ DI ĐỘNG

BRIEFNEWS – APP ĐỌC BÁO

Ngành : Công nghệ thông tin

Niên khoá : 2023-2027

Giảng viên hướng dẫn : Th.S Võ Tấn Toàn Sinh viên
thực hiện :

Phạm Bá Hải Anh 23130334 (NhÃ³m trÆ°á»Ÿng)

Tổng Phan Hoàng Duy 23130258

Võ Sinh 23130087

Ninh Quang Tuấn 23130300

Phan Ngọc Vĩnh

TP.HỒ CHÍ MINH, tháng 4 năm 2026

MỤC LỤC

CHƯƠNG 1.	GIỚI THIỆU	3
1.1.	Bối cảnh.....	3
1.2.	Lí do chọn đề tài.....	3
1.3	Mục tiêu.....	3
1.4	Phạm vi.....	3
1.5	Đối tượng sử dụng.....	3
1.6	Kết quả mong muốn	3
CHƯƠNG 2.	PHÂN TÍCH YÊU CẦU	5
2.1	Bài toán.....	5
2.2	Yêu cầu chức năng	5
2.3	Yêu cầu phi chức năng	5
2.4	Đối tượng sử dụng.....	6
2.5	Use Case và Mô tả Use Case.....	6
2.6	Quy tắc nghiệp vụ	7
CHƯƠNG 3.	THIẾT KẾ HỆ THỐNG	7
3.1	Kiến trúc tổng thể.....	7
3.2	Kiến trúc Mobile	8
3.3	Kiến trúc Backend.....	9
3.4	Kiến trúc Cơ sở dữ liệu	9
3.5	Công nghệ sử dụng.....	9
3.6	Luồng gọi API và Xác thực.....	10
3.7	Lý do lựa chọn kiến trúc.....	10
CHƯƠNG 4.	CÀI ĐẶT TRIỂN KHAI.....	11
4.1	Cấu trúc Project Mobile	11
4.2	Các màn hình chức năng chính	11
4.2.1	Màn hình Trang chủ (HomeFragment).....	11

4.2.2 Màn hình Đánh dấu trang (BookmarkFragment)	12
4.2.3 Màn hình Tìm kiếm (SearchFragment)	13
4.3 Quản lý Trạng thái và Điều hướng	14
4.4 Kỹ thuật gọi API và Lưu trữ dữ liệu	14
4.5 Xử lý lỗi (Error Handling).....	15
4.6 Triển khai Backend.....	15
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	15
5.1 Kết quả đạt được	15
5.2 Ưu điểm	15
5.3 Hạn chế.....	16
5.4 Bài học kinh nghiệm	16
5.5 Hướng phát triển.....	16

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh

Trong kỷ nguyên thông tin số, nhu cầu tiếp cận tin tức nhanh chóng và liên tục từ các thiết bị di động đang trở thành một phần thiết yếu của cuộc sống hàng ngày. Các nền tảng tin tức đa dạng nhưng thường bị phân mảnh, đòi hỏi một giải pháp tập trung để tối ưu hóa trải nghiệm đọc tin.

1.2. Lí do chọn đề tài

Đề tài "Phát triển ứng dụng di động NLU Banking" được lựa chọn nhằm xây dựng một ứng dụng tổng hợp tin tức trên nền tảng Android. Thay vì để ứng dụng di động gọi trực tiếp các nguồn tin tức bên thứ ba (dễ gặp vấn đề về giới hạn lưu lượng và hiệu năng), dự án kết hợp với một hệ thống Backend nội bộ làm lớp đệm, cung cấp dữ liệu ổn định và nhanh chóng cho người dùng.

1.3 Mục tiêu

Xây dựng ứng dụng di động Android trực quan, mượt mà và dễ sử dụng.

Ứng dụng các kiến trúc phần mềm hiện đại của Android (như MVVM, Repository pattern) để đảm bảo mã nguồn dễ bảo trì và mở rộng.

Xây dựng hệ thống Backend để thu thập, phân loại và cung cấp dữ liệu tin tức qua REST API.

1.4 Phạm vi

Ứng dụng Mobile: Nền tảng Android, cung cấp giao diện hiển thị tin tức, tìm kiếm và lưu trữ bài báo ngoại tuyến.

Hệ thống Backend: Triển khai các dịch vụ thu thập tin tức định kỳ và cung cấp REST API cho ứng dụng di động.

1.5 Đối tượng sử dụng

Người dùng sở hữu thiết bị di động Android có nhu cầu cập nhật tin tức hàng ngày một cách tổng hợp và nhanh chóng.

1.6 Kết quả mong muốn

Một ứng dụng Android hoàn chỉnh có khả năng tương tác mượt mà, không gây giật lag (chặn Vòng chính - Main Thread).

Dữ liệu được quản lý hiệu quả thông qua cấu trúc cơ sở dữ liệu cục bộ, cho phép hỗ trợ đọc ngoại tuyến.

Một hệ thống Backend đóng vai trò như lớp cung cấp dữ liệu ổn định và độc lập.

CHƯƠNG 2. PHÂN TÍCH YÊU CẦU

2.1 Bài toán

Hệ thống cần cung cấp cho người dùng một giao diện để lướt tin tức mới nhất, tìm kiếm các bài viết theo chủ đề và cho phép lưu lại các bài viết quan trọng để đọc khi không có kết nối mạng. Để tránh việc ứng dụng di động (Client) gặp giới hạn khi truy cập trực tiếp từ các nguồn tin (ví dụ: NewsAPI), một Backend nội bộ sẽ đóng vai trò tổng hợp dữ liệu, lưu trữ đệm và phân phối lại cho thiết bị di động.

2.2 Yêu cầu chức năng

Mã YC	Tên chức năng	Mô tả chức năng
YC_01	Hiển thị tin tức nổi bật	Tải và hiển thị danh sách tin tức nổi bật nhất trên giao diện chính.
YC_02	Lưu bài báo (Bookmark)	Người dùng có thể đánh dấu trang một bài báo để đọc lại sau.
YC_03	Tìm kiếm tin tức	Cho phép người dùng tìm kiếm bài báo bằng từ khóa.
YC_04	Tải dữ liệu ngoại tuyến	Xem được danh sách các bài báo đã lưu kể cả khi không có kết nối mạng.
YC_05	Cập nhật dữ liệu từ Backend	Backend tự động lấy tin từ các nguồn bên ngoài theo chu kỳ.

2.3 Yêu cầu phi chức năng

Hiệu năng: Các tác vụ gọi mạng (network calls) hoặc truy xuất cơ sở dữ liệu phải được thực hiện trên Luồng chạy ngầm (Background Thread) để không gây nghẽn Luồng chính (blocking Main Thread) hay hiện tượng Ứng dụng không phản hồi (ANR).

Kiến trúc: Tuân thủ nguyên tắc Nguồn sự thật duy nhất (Single Source of Truth), ưu tiên sử dụng dữ liệu từ cơ sở dữ liệu cục bộ thay vì chỉ phụ thuộc vào kết nối mạng.

Độc lập và Phân tách: Các thành phần trong ứng dụng phải có khả năng hoạt động độc lập theo chức năng, dễ dàng bảo trì.

2.4 Đối tượng sử dụng

Người dùng cuối (End-users): Sử dụng ứng dụng Android để đọc và lưu trữ tin tức.

Hệ thống tự động: Scheduler của Backend tự động cập nhật tin tức.

2.5 Use Case và Mô tả Use Case

Biểu đồ Use Case (Mức Mobile)

Người dùng tương tác trực tiếp với ba chức năng chính:

- Xem danh sách tin tức.
- Tìm kiếm tin tức.
- Đánh dấu bài báo.

Mô tả Use Case

Use Case: Xem danh sách tin tức

Mục đích: Người dùng muốn cập nhật các tin tức mới nhất.

Luồng chính:

1. Người dùng mở ứng dụng (mặc định tại màn hình trang chủ).
2. Ứng dụng hiển thị hiệu ứng tải .
3. Hệ thống gọi API để lấy danh sách bài báo mới nhất, hoặc trích xuất từ cơ sở dữ liệu cục bộ.
4. Hệ thống hiển thị danh sách tin tức lên màn hình.

Use Case: Tìm kiếm tin tức (Bookmark)

Mục đích: Người dùng tìm một bài báo để đọc theo từ khóa.

Luồng chính:

1. Người dùng chọn vào thanh tìm kiếm trên màn hình
2. Người dùng nhập từ khóa tìm kiếm bài báo
3. Hệ thống trả ra danh sách các bài báo theo từ khóa

Use Case: Đánh dấu bài báo (Bookmark)

Mục đích: Người dùng lưu một bài báo để đọc ngoại tuyến.

Luồng chính:

1. Người dùng chọn biểu tượng "Bookmark" trên một bài báo.
2. Hệ thống lưu đối tượng bài báo đó xuống cơ sở dữ liệu SQLite cục bộ.
3. Giao diện tự động cập nhật biểu tượng trạng thái lưu thành công.

2.6 Quy tắc nghiệp vụ

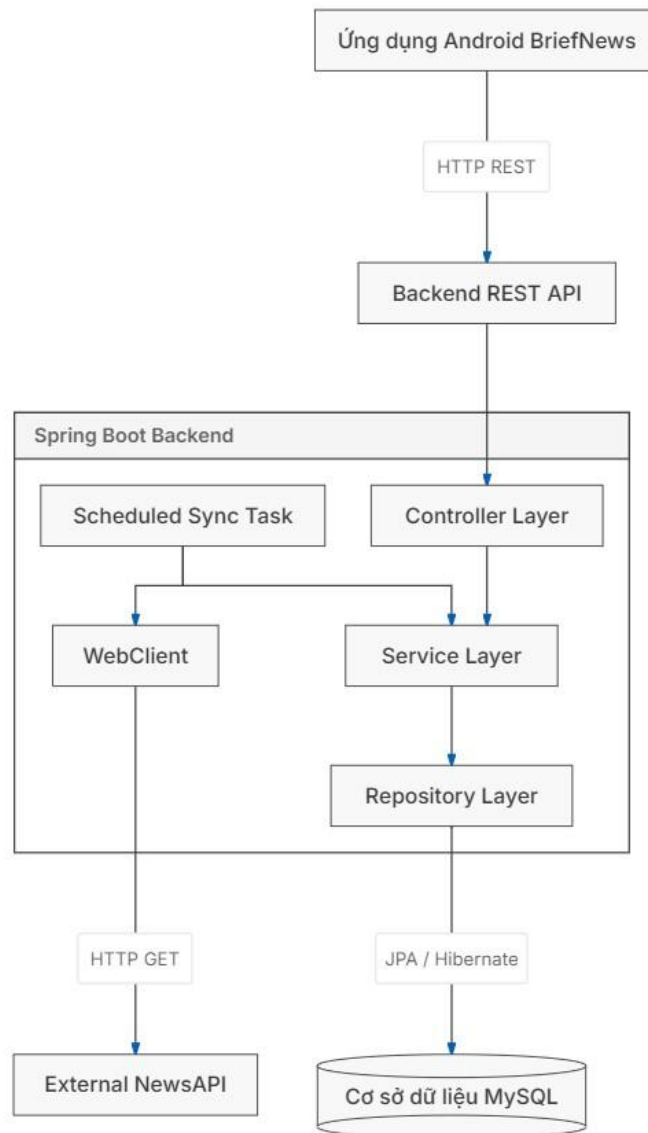
Mobile không kết nối trực tiếp với dịch vụ bên thứ ba mà chỉ tương tác với Backend nội bộ qua REST API.

Dữ liệu trả về luôn sử dụng định dạng JSON được đóng gói chuẩn hóa.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1 Kiến trúc tổng thể

Hệ thống được thiết kế theo mô hình Client-Server. Ứng dụng Android (Client) kết nối với Spring Boot Backend (Server) thông qua giao thức HTTP REST.



3.2 Kiến trúc Mobile

Ứng dụng Android tuân thủ chặt chẽ kiến trúc MVVM (Model-View-ViewModel) kết hợp với Repository pattern.

View (Giao diện): Bao gồm Activity (thành phần giao diện cốt lõi) và Fragment (một phần giao diện tái sử dụng). Chịu trách nhiệm hiển thị dữ liệu và nhận tương tác từ người dùng. Không chứa bất kỳ logic nghiệp vụ nào.

ViewModel (lớp quản lý trạng thái): Đóng vai trò cầu nối giữa View và dữ liệu. Nó lưu trữ trạng thái hiển thị và miễn nhiệm với các thay đổi cấu hình vòng đời của hệ điều hành.

Repository (lớp quản lý dữ liệu): Lớp trung gian quyết định lấy dữ liệu từ mạng (REST API) hay từ cơ sở dữ liệu cục bộ.

Model: Biểu diễn các cấu trúc dữ liệu.

3.3 Kiến trúc Backend

Backend áp dụng N-Tier Layered Architecture (Kiến trúc phân tầng 3 lớp):

Controller: Tiếp nhận và xác thực yêu cầu HTTP từ Mobile. Không chứa logic nghiệp vụ.

Service: Chứa logic nghiệp vụ cốt lõi, chuyển đổi dữ liệu và xử lý nghiệp vụ.

Repository: Giao tiếp với cơ sở dữ liệu thông qua Spring Data JPA/Hibernate.

3.4 Kiến trúc Cơ sở dữ liệu

Cơ sở dữ liệu Mobile: Sử dụng SQLite cục bộ, thông qua thư viện Room (Room Database). Room đóng vai trò lưu trữ các bài báo được người dùng đánh dấu trang, đóng vai trò "Single Source of Truth" (Nguồn sự thật duy nhất).

Cơ sở dữ liệu Backend: Sử dụng MySQL triển khai trên đám mây (Aiven Cloud) để lưu trữ vĩnh viễn tin tức được tổng hợp từ các nguồn tin quốc tế.

3.5 Công nghệ sử dụng

Nền tảng	Công nghệ	Vai trò
Mobile	Kotlin	Ngôn ngữ lập trình chính cho Android.
Mobile	ViewModel & LiveData	Quản lý trạng thái và dữ liệu giao diện theo tính chất phản ứng (reactive).
Mobile	Coroutine	Xử lý bất đồng bộ (asynchronous), chuyển đổi luồng không gây chặn (non-blocking).
Mobile	Retrofit	Xây dựng các yêu cầu HTTP giao tiếp với REST API.
Mobile	Room	Quản lý lưu trữ SQLite cục bộ.
Mobile	Navigation Component	Quản lý điều hướng giữa các Fragment.
Backend	Spring Boot	Framework cốt lõi để xây dựng REST API.
Backend	Hibernate / JPA	ORM để giao tiếp với MySQL.

Backend	Jackson	Chuyển đổi đối tượng Java thành định dạng JSON.
---------	---------	---

3.6 Luồng gọi API và Xác thực

Luồng lấy tin tức diễn ra như sau:

- 1.Fragment yêu cầu dữ liệu từ ViewModel.
- 2.ViewModel sử dụng Coroutine để gọi phương thức suspend từ Repository trên luồng chạy ngầm.
- 3.Repository kích hoạt Retrofit để thực hiện truy vấn HTTP GET tới Backend.
- 4.JSON trả về từ Backend được Retrofit chuyển đổi thành đối tượng dữ liệu.
- 5.Repository có thể lưu dữ liệu xuống Room, sau đó chuyển trả về ViewModel để cập nhật giao diện (View).

Về luồng xác thực: Hệ thống Backend được thiết kế chủ đích với các API công khai (Public API) do dữ liệu cung cấp (tin tức) là thông tin công cộng. Quá trình xác thực người dùng (Authentication) và quản lý tài khoản được ủy quyền hoàn toàn cho ứng dụng Mobile (thông qua Firebase Authentication). Các tính năng cá nhân hóa như Đánh dấu trang (Bookmark) được quản lý ở cơ sở dữ liệu cục bộ trên thiết bị di động.

3.7 Lý do lựa chọn kiến trúc

Sự phân tách trách nhiệm (Separation of Concerns): Việc tách biệt rõ ràng giữa UI, trạng thái (ViewModel) và truy xuất dữ liệu (Repository) giúp mã nguồn dễ đọc và tạo điều kiện thuận lợi cho việc kiểm thử. Việc sử dụng ViewModel giúp duy trì trạng thái an toàn qua các thay đổi vòng đời ứng dụng (configuration changes) mà không giữ tham chiếu mạnh tới Context/Activity, từ đó ngăn ngừa lỗi rò rỉ bộ nhớ (Memory Leak), trong khi Repository đóng vai trò trừu tượng hóa các nguồn dữ liệu.

Hiệu suất: Sử dụng Coroutine và Room tối ưu hoá hiệu năng, đảm bảo Luồng chính (Main Thread) không bao giờ bị nghẽn bởi các tác vụ mạng hay đĩa cứng.

CHƯƠNG 4. CÀI ĐẶT TRIỂN KHAI

4.1 Cấu trúc Project Mobile

Cấu trúc ứng dụng được tổ chức theo kiến trúc lai "Tính năng-Lớp" (Feature-Layer hybrid architecture):

vn.edu.hcmuaf.fit.nlu banking.ui: Chứa bộ điều khiển giao diện. Không chứa logic gọi API hay database.

- fragments/: Các màn hình cơ bản như HomeFragment, SearchFragment, ProfileFragment.
- adapters/: Chứa NewsAdapter để gán dữ liệu vào danh sách.

vn.edu.hcmuaf.fit.nlu banking.viewmodel: Nơi chứa NewsViewModel, quản lý LiveData để tương tác với UI.

vn.edu.hcmuaf.fit.nlu banking.data: Xử lý toàn bộ logic dữ liệu.

- api/: Cấu hình RetrofitClient và định nghĩa các endpoint NewsApiService.
- database/: Chứa cấu hình NewsDatabase và ArticleDao.
- repository/: NewsRepository điều phối dữ liệu mạng và dữ liệu cục bộ.
- model/: Các cấu trúc đối tượng (như Article).

4.2 Các màn hình chức năng chính

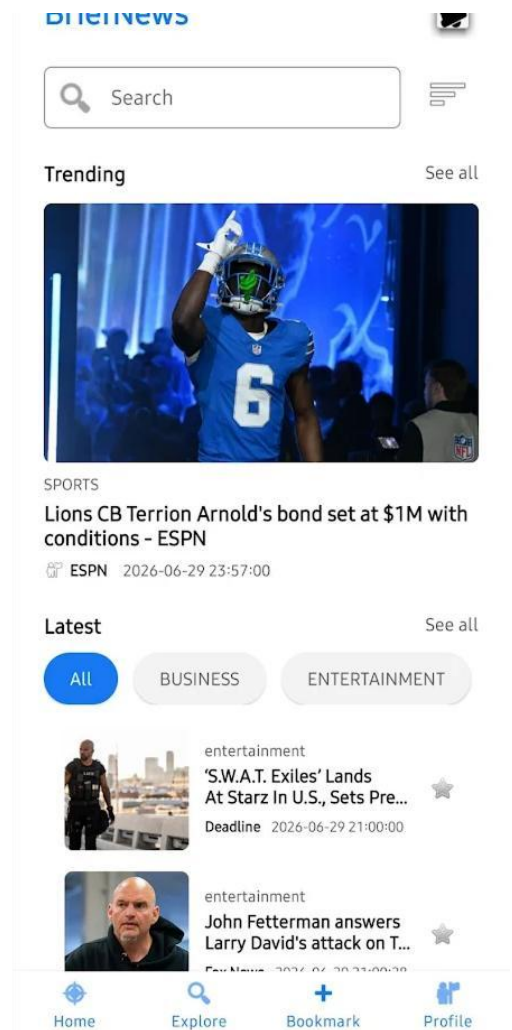
4.2.1 Màn hình Trang chủ (HomeFragment)

Mục đích: Hiển thị bảng tin chính cho người dùng ngay khi khởi chạy.

Luồng xử lý: HomeFragment quan sát LiveData từ NewsViewModel. Nếu chưa có dữ liệu, hiệu ứng khung xương (Shimmer) được bật. ViewModel tiến hành gọi Repository để tải danh sách bài báo từ Backend qua Retrofit. Khi nhận được phản hồi, dữ liệu được chuyển cho NewsAdapter đưa vào RecyclerView.

Công nghệ sử dụng: ViewModel, LiveData, Retrofit, Glide (để tải hình ảnh), ViewBinding.

API liên quan: GET /api/articles.

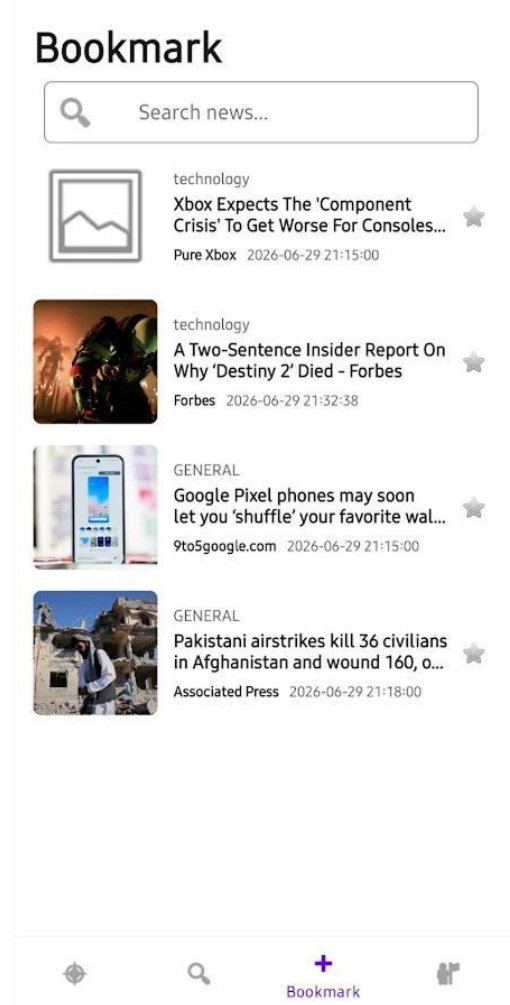
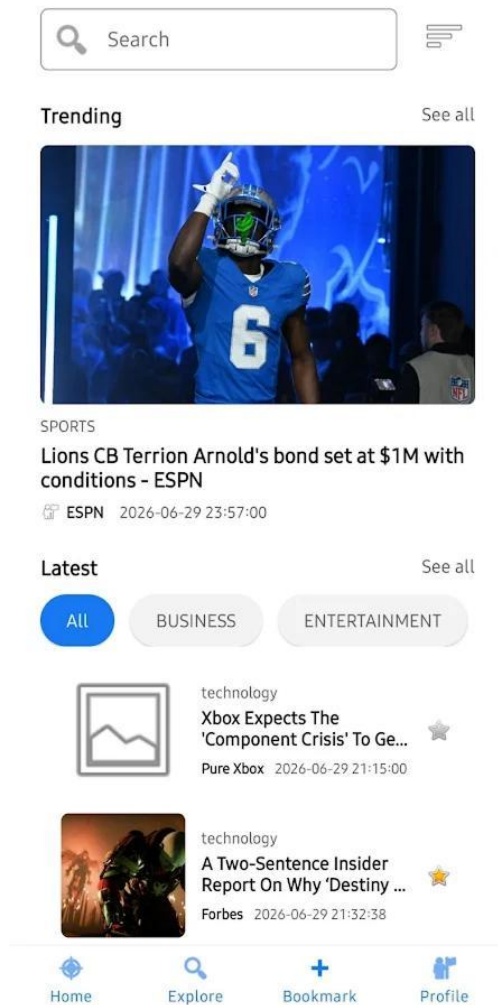


4.2.2 Màn hình Đánh dấu trang (BookmarkFragment)

Mục đích: Xem các tin tức đã lưu ngoại tuyến.

Luồng xử lý: Khi người dùng nhấn nút lưu hình ngôi sao trong HomeFragment, đối tượng Article được chèn vào SQLite qua DAO. Hành động này tự động kích hoạt tính phản ứng của Room, cập nhật lại dòng dữ liệu LiveData<List<Article>>. BookmarkFragment lập tức nhận được luồng dữ liệu mới và hiển thị qua Adapter.

Công nghệ sử dụng: Room Database, Coroutine (suspend function cho tác vụ INSERT), LiveData truy vấn trực tiếp từ cơ sở dữ liệu.

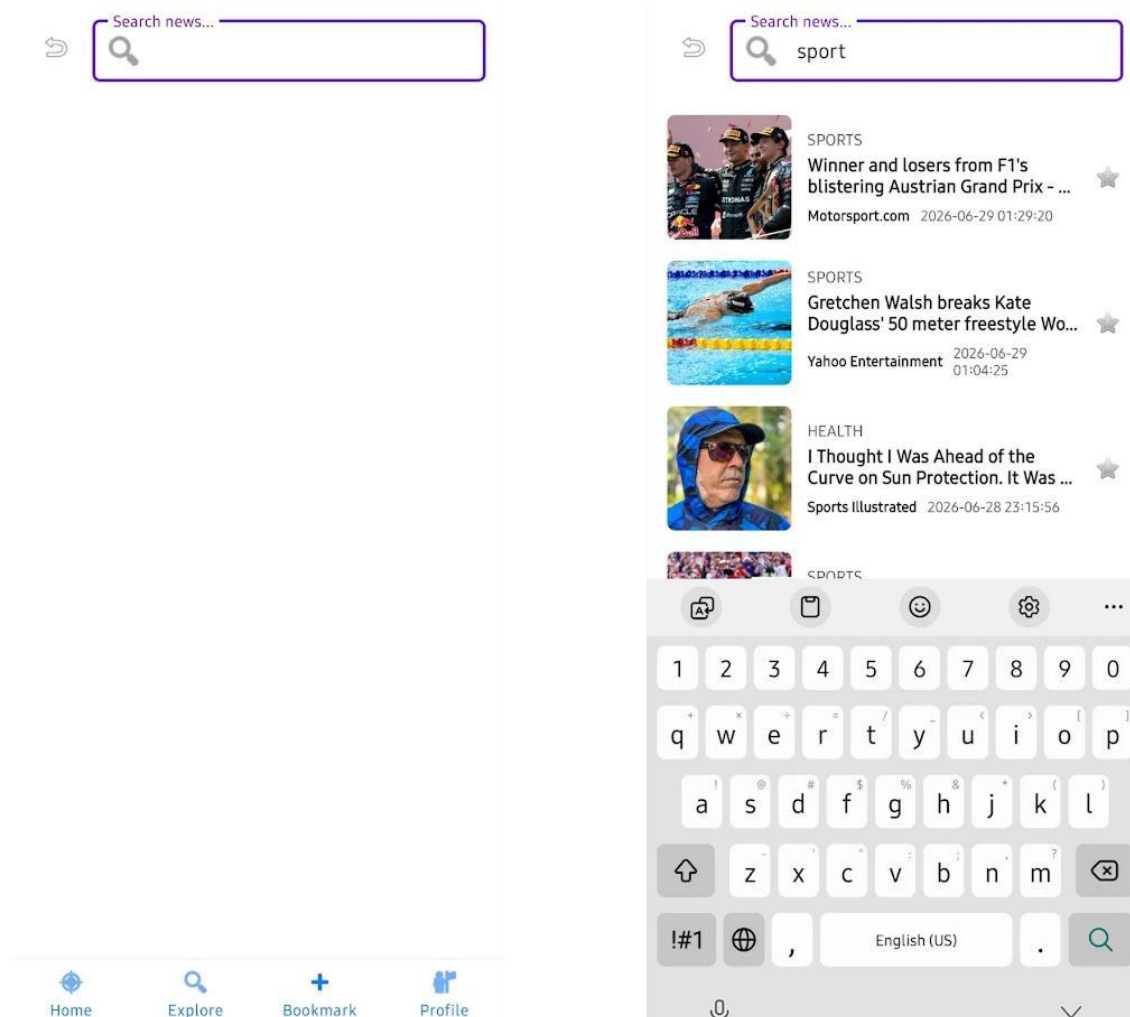


4.2.3 Màn hình Tìm kiếm (SearchFragment)

Mục đích: Tìm kiếm tin tức theo từ khóa và hỗ trợ tải thêm dữ liệu khi cuộn.

Luồng xử lý: Khi Fragment được khởi tạo, RecyclerView được gắn với NewsAdapter và LinearLayoutManager. SearchFragment gọi viewModel.searchArticles (query, page, pageSize) để lấy dữ liệu. Kết quả được quan sát qua LiveData<Resource <NewsResponse>>: trạng thái SUCCESS cập nhật Adapter và kiểm tra trang cuối, trạng thái ERROR hiển thị thông báo lỗi. Khi người dùng cuộn đến cuối danh sách, PagingScrollListener kích hoạt loadNextPage() để tăng số trang và tải thêm dữ liệu. Khi chọn một bài báo, URL sẽ được mở bằng Custom Tabs.

Công nghệ sử dụng: MVVM (ViewModel + LiveData), RecyclerView, Handler debounce, phân trang thủ công (Pagination), Custom Tabs.



4.3 Quản lý Trạng thái và Điều hướng

Điều hướng: Ứng dụng dùng cấu trúc Activity đơn (MainActivity) chứa NavHostFragment. Việc chuyển đổi giữa Home, Search và Bookmark được xử lý bởi Navigation Component, giúp đồng bộ hóa các luồng đi lại an toàn mà không phải thủ công tạo Fragment.

Quản lý trạng thái: Áp dụng triệt để LiveData. Giao diện người dùng sẽ phản ứng và tự vẽ lại (reactive UI) dựa trên sự biến động dữ liệu được cung cấp từ ViewModel.

4.4 Kỹ thuật gọi API và Lưu trữ dữ liệu

Gọi API: Xây dựng cấu hình Retrofit kết hợp cùng OkHttp. Sử dụng Gson để tự động phân tích cú pháp (JSON parsing) dữ liệu từ Backend thành các đối tượng Article trong Kotlin thông qua tính năng Reflection.

Lưu trữ dữ liệu: Room Database được sử dụng để che giấu các câu lệnh SQL truyền thống. Repository điều phối các thao tác truy vấn lưu, đọc, xoá.

4.5 Xử lý lỗi (Error Handling)

Để ngăn ngừa treo ứng dụng khi có lỗi mạng:

Các tác vụ mạng được bọc trong các Coroutine xử lý bất đồng bộ.

Repository phân tách rõ ràng việc truy vấn nếu thất bại sẽ ném ra ngoại lệ (Exception) hoặc cảnh báo lỗi lên UI thông qua trạng thái (State).

4.6 Triển khai Backend

Backend hoạt động trên nền tảng Java Spring Boot.

Cơ sở dữ liệu MySQL được triển khai thông qua dịch vụ đám mây Aiven Cloud.

Được thiết lập một Cron Scheduler (NewsSyncScheduler) tự động đánh thức hệ thống để đồng bộ dữ liệu mới nhất từ NewsAPI, đảm bảo Mobile luôn nhận được nội dung cập nhật mới.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả đạt được

Dự án đã xây dựng thành công bộ khung hệ thống hoàn chỉnh cho ứng dụng tin tức NLU Banking, bao gồm:

Ứng dụng Android (Mobile Client) tích hợp thành thạo các thư viện và kỹ thuật hiện đại như Navigation Component, Room, Retrofit và Coroutine.

Backend Spring Boot hỗ trợ tốt trong vai trò trạm trung chuyển dữ liệu và lưu trữ, tối ưu cho ứng dụng di động thay vì bắt ứng dụng di động phải kết nối trực tiếp đến các dịch vụ bên ngoài.

5.2 Ưu điểm

Kiến trúc rõ ràng: Việc phân chia gói mã nguồn Mobile theo cấu trúc lớp và MVVM giúp mã nguồn rõ ràng, dễ bảo trì, dễ thay đổi.

Tối ưu trải nghiệm (UX): Chuyển đổi các tác vụ nặng xuống luồng nền thông qua Coroutine giúp tối ưu hóa hiệu năng, giảm thiểu tình trạng nghẽn Luồng chính (Main Thread) và duy trì độ phản hồi của giao diện.

Cơ chế ngoại tuyến (Offline): Ứng dụng hỗ trợ trải nghiệm đọc bài đã đánh dấu (bookmark) ngay cả khi không có kết nối mạng thông qua việc đồng bộ cơ sở dữ liệu cục bộ.

5.3 Hạn chế

Quá trình xác thực người dùng (Authentication) và đồng bộ danh sách đã lưu lên đám mây cá nhân chưa được triển khai đầy đủ.

Thiếu các bộ kiểm thử tự động (Automated Testing) toàn diện cho cả Mobile lẫn Backend.

5.4 Bài học kinh nghiệm

Sự phụ thuộc và liên kết chặt chẽ (tight-coupling) trong lập trình ứng dụng di động luôn dẫn đến các lỗi khó quản lý (ví dụ: truyền Activity Context sai chỗ gây rò rỉ bộ nhớ - Memory Leak). Do đó, nguyên tắc phân tách trách nhiệm (Separation of Concerns) là yếu tố mang tính sống còn đối với ứng dụng Android.

Ứng dụng di động nên hạn chế gọi các API bên thứ ba không ổn định, thay vào đó việc xây dựng một Backend nội bộ làm lớp đệm mang lại lợi ích lớn trong việc tự kiểm soát luồng dữ liệu, định dạng JSON và bảo mật.

5.5 Hướng phát triển

Hoàn thiện cơ chế đăng nhập, đăng ký và xác thực người dùng (ví dụ: tích hợp JWT vào Backend).

Đồng bộ hóa dữ liệu Bookmark giữa thiết bị và máy chủ, giúp người dùng lưu trạng thái trên nhiều thiết bị khác nhau.

Xây dựng hệ thống Kiểm thử tự động (Unit Test / Integration Test) đảm bảo chất lượng hệ thống trước những thay đổi trong tương lai.